# Measuring the Defenders: A Layer-Aware, Framework-Mapped Benchmark for Model Context Protocol Security Proxies

Gowthaman Arumugam, Independent Researcher (agowthaman90@gmail.com)

July 13, 2026 · preprint (not peer-reviewed)

## Contents



*Artifacts: mcp-defense-bench (benchmark) · mcp-bastion (reference proxy). Data CC-BY-4.0; code Apache-2.0.* ## Abstract

The Model Context Protocol (MCP) has become a common way to connect large-language-model (LLM) agents to external tools and data, and a substantial 2025–2026 literature now documents its attack surface. Existing security benchmarks for MCP measure how well an **LLM agent resists** attacks. None, to our knowledge, measure how well a **defensive proxy or gateway covers** that attack surface, and none map their results to the governance frameworks organizations actually use — the NIST AI Risk Management Framework (AI RMF) and the OWASP Top 10 for LLM and Agentic Applications. We present two artifacts to fill this gap: (1) a **threat–control crosswalk** of 22 MCP attack vectors mapped to architectural layer, STRIDE class, NIST AI RMF function, and OWASP category; and (2) an open, vendor-neutral **defense-side benchmark** that runs a candidate MCP security proxy against a corpus of attack fixtures with matched benign controls, and scores its coverage using the crosswalk as a rubric. Each tool is driven through its *real* code, not a mock. We evaluate three open-source proxies (mcp-bastion, mcp-firewall, pipelock) plus a null baseline over 28 test cases. The strongest proxy reaches **34% weighted coverage (13 of 22 vectors)** — a proxy's realistic ceiling — and even the union of all three tools leaves **9 of 22 vectors covered by none**: the entire registry/supply-chain surface, OS-isolation-dependent vectors, and several semantic vectors. All results carry zero false positives against matched benign controls. We argue this is direct, quantified evidence that a runtime proxy alone is insufficient and that defense-in-depth spanning *distinct mechanism classes* (proxy, cryptographic attestation, OS isolation) is

required. We also report two methodological findings we observed first-hand: benchmark scores are sensitive to how attack fixtures are encoded (a fairness hazard), and the benchmark can *guide* a defender's development — over the study, mcp-bastion's coverage rose from 9% to 34% as gaps it surfaced were fixed and re-verified. A matched benign-control design and a "measure only what the tool inspects at runtime" integrity rule keep the results honest.

---

## 1. Introduction

MCP standardizes how an LLM agent discovers and calls external "tools" exposed by "servers." Its rapid adoption across consumer and enterprise agents has been accompanied, in 2025–2026, by a dense body of security research: systematization-of-knowledge papers, formal threat taxonomies, attack benchmarks, and cryptographic attestation proposals (Section 2). This work establishes *what can go wrong*.

A practical question remains under-addressed: **given a defensive tool that sits in front of MCP servers — a proxy, gateway, or scanner — how much of the known attack surface does it actually cover?** Answering it well requires three things that, individually, exist in the literature but have not been combined:

1. a **shared taxonomy** of MCP attack vectors;
2. a mapping from those vectors to the **governance frameworks** (NIST AI RMF, OWASP) that security and procurement teams reason in; and
3. an **executable, reproducible** way to test a defender and report coverage.

We contribute all three, plus an empirical study. Concretely:

- **A threat–control crosswalk** (Section 3): 22 MCP attack vectors, each mapped to one or more of six architectural layers, a STRIDE class, NIST AI RMF functions, and OWASP LLM-2025 and Agentic-2026 categories. Published as open, versioned, machine-readable data.
- **A defense-side benchmark** (Section 4): for each vector, one or more attack **fixtures** paired with a near-identical **benign control**; a small **adapter** per tool that drives the tool's real detection code; and a scorer that reports coverage per layer and per framework.
- **An empirical evaluation** (Sections 5–6) of three open-source proxies and a null baseline, yielding the complementarity finding above.
- **A candid methodology discussion** (Sections 7–8): the corpus-encoding sensitivity we observed, the matched-control and runtime-integrity safeguards, and the limits of the study.

Our aim is not to rank products but to map coverage, expose gaps, and provide reusable infrastructure.

## 2. Related Work

> All 2603/2604-series works below are recent, non-peer-reviewed preprints whose taxonomies are their authors' own constructs; we cite them as *proposed by*, not as established consensus. Coverage statistics quoted from any single paper reflect that paper's surveyed set.

**Threat taxonomies.** Recent work systematizes MCP threats through several lenses: an SoK separating adversarial security threats (indirect prompt injection, tool poisoning) from epistemic safety hazards across the Resources/Prompts/Tools primitives [SoK-2512.08290]; a formal framework of

7 threat categories and 23 attack vectors grounded in a large tool corpus [Formal-2604.05969]; a STRIDE/DREAD model over five MCP components that identifies tool poisoning as the most impactful client-side vulnerability [STRIDE-2603.22489]; and a defense-placement taxonomy classifying defenses by the layer that enforces them, which finds protection "uneven and predominantly tool-centric" with weak transport, host-orchestration, and supply-chain coverage [MCP-DPT-2604.07551]. Our crosswalk reuses these vector families but adds the framework mapping none of them provides.

**Attack-side benchmarks.** MSB [MSB-2510.15994] (12 attacks, ~2,000 instances, real MCP execution) measures the resistance of the **LLM agent** across the tool-use pipeline; MCPTox [MCPTox-2508.14925] scopes to tool poisoning; AgentDefense-Bench aggregates datasets into MCP-JSON-RPC test cases; MCP-Bench [MCPBench-2508.20453] measures agent *capability*, not security. All measure the agent (or the model), not a **defensive proxy**. Ours measures the defender, on the full mapped surface, and reports coverage crosswalked to NIST/OWASP — which we could not find in any surveyed benchmark.

**Attestation and provenance.** ETDI [ETDI-2506.01333] proposes signed, versioned tool definitions; a "Trustworthy MCP Registry" blueprint [TrustReg-FI2026] composes existing standards (well-known URIs, Sigstore, JSON Canonicalization + JWS) for provenance and runtime integrity, and notes the official MCP registry remains an unverified pointer architecture. These are complementary to our benchmark; a future attestation-conformance suite (Section 9) would test them.

**Scanners and gateways.** Practitioner tools include Invariant Labs' MCP-Scan (since acquired by Snyk and now a cloud-gated scanner), Lasso's and EnkryptAI's MCP gateways, Docker's isolation-based MCP gateway, and the proxies we evaluate here. Our benchmark scores locally-deterministic defenders and treats cloud-model defenders as observable-but-not-reproducibly-scoreable (Section 4.4).

## 3. The Threat–Control Crosswalk

The crosswalk is the benchmark's rubric and a standalone artifact. It enumerates **22 MCP attack vectors** consolidated from the taxonomies in Section 2, and maps each to:

- **Architectural layer** (one of: host-orchestration, client, transport, server, tool, registry-supply-chain) — following the defense-placement view of [MCP-DPT-2604.07551];
- **STRIDE** class;
- **NIST AI RMF** function(s): Govern, Map, Measure, Manage;
- **OWASP Top 10 for LLM Applications (2025)** category (LLM01–LLM10); and
- **OWASP Top 10 for Agentic Applications (2026)** category (ASI01–ASI10).

Table 1 lists the vectors and their primary layer. The full mapping, including STRIDE and all framework codes, is published as machine-readable data (`rubric/crosswalk.json`, CC-BY-4.0).

**Table 1. The 22 vectors (primary layer).**

| # | Vector | Layer | # | Vector | Layer |
|---|---|---|---|---|---|
| 1 | Tool poisoning | tool | 12 | Configuration drift | server |

| # | Vector | Layer | # | Vector | Layer |
|---|---|---|---|---|---|
| 2 | Tool shadowing / name collision | client | 13 | Sandbox escape | server |
| 3 | Rug pull (definition mutation) | tool | 14 | Schema / validation bypass | server |
| 4 | Out-of-scope parameter injection | tool | 15 | Man-in-the-middle (transport) | transport |
| 5 | Prompt injection via tool results | tool | 16 | DNS rebinding (local servers) | transport |
| 6 | Indirect / retrieval injection | tool | 17 | Server impersonation | registry-supply-chain |
| 7 | Cross-tool exfiltration (confused deputy) | client | 18 | Excessive permission / escalation | host-orchestration |
| 8 | Tool-transfer / cross-server chaining | host-orchestration | 19 | Credential / token theft | host-orchestration |
| 9 | False-error escalation | tool | 20 | Consent fatigue / over-broad grants | client |
| 10 | Package / name squatting | registry-supply-chain | 21 | Command injection | server |
| 11 | Supply-chain poisoning (provenance gap) | registry-supply-chain | 22 | System-prompt / context leakage | client |

The mappings are indicative and reviewable, not a certification — we invite community correction, which is itself a goal of publishing the rubric openly. The OWASP Agentic categories used are the official 2026 set: ASI01 Agent Goal Hijack, ASI02 Tool Misuse and Exploitation, ASI03 Identity and Privilege Abuse, ASI04 Agentic Supply Chain Vulnerabilities, ASI05 Unexpected Code Execution (RCE), ASI06 Memory & Context Poisoning, ASI07 Insecure Inter-Agent Communication, ASI08 Cascading Failures, ASI09 Human-Agent Trust Exploitation, ASI10 Rogue Agents.

## 4. Benchmark Methodology

### 4.1 Design

For each vector the corpus contains one or more **test cases**. A test case is a static JSON object with: a **malicious fixture** (the attack, as data — a tool definition, tool call, tool result, registry entry, or scenario), a near-identical **benign control**, the **expected** defender behavior (`detect`/`enforce`), and a **provenance** note. Fixtures are static data, never live exploits; attacker hosts use reserved names (`attacker.example`; the cloud-metadata address `169.254.169.254` appears only as an SSRF *target* in data, never contacted).

### 4.2 Adapters and the runtime-integrity rule

Each evaluated tool provides a thin **adapter** exposing `assess(input) → {detect, enforce}`. The runner calls it once on the malicious fixture and once on the benign control. Adapters drive the tool's **real** code (e.g., importing the proxy's detection functions, or invoking its CLI/SDK with a committed configuration).

A central rule keeps scores honest:

> **Report only what the tool actually inspects at runtime — never what its rules *could* match if pointed at data the tool never sees.**

For example, a proxy that inspects tool *definitions* but not tool *results* returns "not detected" for result-borne attacks, even if its text rules would match the result string in isolation. This measures deployed behavior, not theoretical capability.

### 4.3 Scoring

Per test case: if the malicious fixture is not flagged, the vector scores **none**; if it is flagged but the benign control is *also* flagged, the vector scores **none** and is marked a **false positive**; if the malicious fixture is flagged and the benign control is clean, the vector scores **detect** (warn) or **enforce** (block). A vector's level is the best across its test cases. Weighted coverage assigns enforce = 1.0, detect = 0.5, none = 0, over 22 vectors. The matched benign control is essential: without it, a tool that flags everything would score perfectly while being useless.

### 4.4 Reproducibility rule

A defender is **scored** only if its detection logic runs **locally and deterministically**. Tools that delegate detection to a proprietary cloud model can be *observed* but not reproducibly *scored*, since their verdicts depend on a black box that can change between runs. This excludes, e.g., the cloud-gated Snyk agent-scan; it includes the three proxies below.

## 5. Experimental Setup

We evaluate four adapters over a 28-case corpus (22 base cases + 6 realistic "v2" cases added for fairness; see Section 7):

| Tool | Class | License | How driven |
|---|---|---|---|
| **null-baseline** | control | — | returns "not detected" for all inputs |

| Tool | Class | License | How driven |
|---|---|---|---|
| **mcp-bastion** | runtime proxy | Apache-2.0 | imports real detection: `scanTool`, `scanText`, `validateArguments`, `checkRequestedScopes`, `checkTransportSecurity`, `checkRequestOrigin`, `hashToolDefinition` |
| **mcp-firewall** | runtime proxy (Python) | AGPL-3.0 | real SDK (`Gateway.check` / `scan_response`) with a committed config |
| **pipelock** | egress firewall (Go) | Apache-2.0 core | real scanner via `explain --json` (offline, deterministic) |

For AGPL/other licensed tools we *run* the tool to benchmark it and report scores; we do not redistribute their code. Each tool is pinned to a released version and configured with its own recommended/starter policy, committed to the repository for reproducibility.

## 6. Results

**Table 2. Verified coverage (28 cases; weighted over 22 vectors).**

| Tool | Class | Weighted coverage | enforce | detect | none | False positives |
|---|---|---|---|---|---|---|
| mcp-bastion | runtime proxy | **34% (7.5/22)** | 2 | 11 | 9 | 0 / 28 |
| mcp-firewall | runtime proxy | **14% (3.0/22)** | 3 | 0 | 19 | 0 / 28 |
| pipelock | egress firewall | **5% (1.0/22)** | 1 | 0 | 21 | 0 / 28 |
| null-baseline | control | 0% | 0 | 0 | 22 | 0 / 28 |

**A proxy's ceiling is partial.** mcp-bastion, the broadest tool, spans the definition, response, argument, transport, and egress/least-privilege layers (poisoning, shadowing, rug-pull; response and retrieval injection and result-borne leakage; schema/parameter validation; plaintext-transport and DNS-rebinding defense; sensitive-argument and over-broad-scope detection). mcp-firewall and pipelock add *enforcement* (deny/redact/SSRF-block) on the call and egress layers where bastion only warns.

**Defense-in-depth across mechanism classes is required.** Even with the strongest proxy at 34%, and across all three tools, **13 of 22 vectors are covered by at least one tool; 9 are covered by none** — the registry/supply-chain surface (package squatting, provenance-gap supply-chain poisoning, server impersonation), OS-isolation-dependent vectors (sandbox escape),

and semantic vectors (tool-transfer, false-error escalation, configuration drift, command injection, consent fatigue). These are not addressable by *any* runtime proxy; they require different mechanisms — cryptographic attestation, OS sandboxing, and client-side controls. A proxy is necessary but not sufficient.

**Zero false positives.** No tool flagged any benign control, across all 28 cases and every feature iteration.

## 7. Discussion

**Benchmark-guided improvement.** The benchmark initially scored mcp-bastion at 9%, covering only the tool-definition layer. It then guided four rounds of proxy-native hardening, each re-measured at zero false positives: response scanning (9%→18%: response/retrieval injection, result-borne leakage), argument/schema validation (18%→23%: parameter smuggling, validation bypass), transport hardening (23%→30%: plaintext-transport warning and a DNS-rebinding Origin check that *enforces* by rejecting cross-origin requests to a loopback listener), and sensitive-argument plus least-privilege scanning (30%→34%: cross-tool exfiltration, over-broad scopes). The benchmark did not just measure the tool; it identified concrete, layer-specific gaps and verified each fix — and then defined the proxy's ceiling, since the remaining 9 vectors are architecturally outside a proxy's reach.

**Corpus-encoding sensitivity (a fairness hazard).** We observed that scores are sensitive to how an attack is *encoded*. An early corpus expressed exfiltration with sanitized placeholder text; against it, mcp-firewall scored 5%. Adding realistic encodings (an AWS-key-shaped secret in a tool result; an exfiltration POST to the cloud-metadata address) raised it to 14% — not by changing the tool, but by probing capabilities the first corpus never exercised. We therefore treat *tool-neutral, realistic, multi-encoding* fixtures as a fairness requirement, and we caution that **absolute totals are corpus- dependent; the per-vector matrix (which vectors a tool covers) is the more reliable read.**

**Adapter faithfulness.** The matched benign control also guards against adapter over-reach. In wiring pipelock (an egress firewall), an initial adapter synthesized an egress URL from an *inbound* local host, causing pipelock to "block" both the malicious and benign case — surfaced immediately as a false positive and corrected by restricting URL synthesis to outbound endpoints.

## 8. Limitations and Threats to Validity

- **Taxonomy is a synthesis, not a standard.** The 22-vector set consolidates several proposed taxonomies; it is one reviewable synthesis. Framework mappings are indicative, single-author, and pending a second-reviewer pass; OWASP Agentic ASI05–ASI10 labels are unconfirmed.
- **Small, seeded corpus.** Most vectors have 1–2 fixtures. Absolute coverage numbers should be read as lower bounds and are corpus-dependent (Section 7).
- **Partial tool interfaces.** pipelock's text-injection scanner (server/eval mode) is not driven; only its URL/egress scanner is measured, so its number is a lower bound on its egress capability.
- **Configuration dependence.** Results depend on each tool's configured policy; we commit the exact configs, but different policies would yield different numbers.
- **Scope.** We score locally-deterministic proxies; cloud-model and isolation-based tools are out of scope for scoring (Section 4.4).

## 9. Future Work

Expand the corpus to multiple tool-neutral fixtures per vector; drive pipelock's content-scanning mode and add further defenders (including a fully-local mode of cloud-gated gateways, if exposed); publish a public leaderboard; complete the second-reviewer pass on framework mappings and confirm ASI labels; and build an **attestation-conformance** suite to test tool-integrity/provenance schemes (ETDI, the Trustworthy Registry composition) as a coupled bench-plus-attestation artifact.

## 10. Conclusion

We introduced a layer-aware, framework-mapped crosswalk of 22 MCP attack vectors and an open, vendor-neutral benchmark that measures how much of that surface a defensive proxy actually covers, driving each tool through its real code. The strongest of three open-source proxies reaches 34% coverage, and 9 of 22 vectors are undefended by any measured tool — the registry/supply-chain, OS-isolation, and semantic vectors that lie outside a proxy's reach. The evidence argues for layered defense across distinct mechanism classes, and the benchmark demonstrably guided one proxy from 9% to 34% at zero false positives. It provides a reusable, honest instrument — with matched benign controls, a runtime-integrity rule, and a reproducibility rule — for the community to measure MCP defenders and track their progress.

## Availability

- **Benchmark (mcp-defense-bench):** rubric, corpus, adapters, runner, leaderboard. Rubric/data: CC-BY-4.0; harness: Apache-2.0.
- **Reference proxy (mcp-bastion):** Apache-2.0; published on npm.
- Evaluated third-party tools are used under their own licenses (mcp-firewall: AGPL-3.0; pipelock: Apache-2.0 core) and are not redistributed here.
- **mcp-defense-bench:** https://github.com/Gowthaman90/mcp-defense-bench
- **mcp-bastion:** https://github.com/Gowthaman90/mcp-bastion · npm: `mcp-bastion`

## References

*Titles, authors, and identifiers below were verified against primary sources on 2026-07-13. The 2603/2604-series works are recent, non-peer-reviewed preprints and may have updated versions; verify current version and page/DOI details at camera-ready time.*


- [SoK-2512.08290] S. Gaire, S. Gyawali, S. Mishra, S. Niroula, D. Thakur, U. Yadav. "Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem." arXiv:2512.08290, Dec. 2025.
- [Formal-2604.05969] N. Acharya, G. K. Gupta. "A Formal Security Framework for MCP-Based AI Agents: Threat Taxonomy, Verification Models, and Defense Mechanisms." arXiv:2604.05969, Apr. 2026.
- [STRIDE-2603.22489] C. Huang, X. Huang, N. P. Tran, A. Milani Fard. "Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning." arXiv:2603.22489, Mar. 2026.

- [MCP-DPT-2604.07551] M. Rostamzadeh, S. Narula, N. Birhan, M. Ghasemigol, D. Takabi. "MCP-DPT: A Defense-Placement Taxonomy and Coverage Analysis for Model Context Protocol Security." arXiv:2604.07551, Apr. 2026.
- [MSB-2510.15994] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, W. Xu. "MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents." arXiv:2510.15994, Oct. 2025 (ICLR 2026).
- [MCPTox-2508.14925] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, X. Li. "MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers." arXiv:2508.14925, Aug. 2025.
- [MCPBench-2508.20453] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, E. Siow. "MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers." arXiv:2508.20453, Aug. 2025.
- [ETDI-2506.01333] M. Bhatt, V. S. Narajala, I. Habler. "ETDI: Mitigating Tool Squatting and Rug Pull Attacks in Model Context Protocol (MCP) by using OAuth-Enhanced Tool Definitions and Policy-Based Access Control." arXiv:2506.01333, Jun. 2025.
- [TrustReg-FI2026] L. Mas, J. Vilaplana, J. Rius, R. Spaimoc, J. Mateo. "The Trustworthy Model Context Protocol (MCP) Registry: An Architectural Blueprint for Cryptographic Provenance and Runtime Integrity." Future Internet, vol. 18, no. 5, art. 243, 2026. doi:10.3390/fi18050243.
- [NIST-AI-RMF] National Institute of Standards and Technology. "AI Risk Management Framework (AI RMF 1.0)." NIST AI 100-1, 2023 (functions: Govern, Map, Measure, Manage).
- [OWASP-LLM-2025] OWASP GenAI Security Project. "OWASP Top 10 for LLM Applications," 2025 edition.
- [OWASP-ASI-2026] OWASP GenAI Security Project. "OWASP Top 10 for Agentic Applications," 2026 (published Dec. 9, 2025; categories ASI01–ASI10).
- [AgentDefense-Bench] A. Sanna. "AgentDefense-Bench" (open-source dataset), GitHub.
- [Snyk-agent-scan] Snyk. "agent-scan" (formerly Invariant Labs MCP-Scan), GitHub / vendor tooling.